

UNIDAD 2 - LA VPS: APROVISIONAR, ENTRAR Y CERRAR

Taller de aula: cerrar un servidor sin tener el servidor

Programación 3 - Tecnicatura Universitaria en Programación - UTN FRM

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener el VPS provisionado. Tampoco hace falta internet. Solo el apunte del capítulo 2 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (25) + 2 (15) + 3 (20) + 4 (15) + 5 (15) + 6 (10).
Que se evalua	Criterio operativo: decidir que se expone y que no, y poder justificar cada decision con el mecanismo que la explica.

Integrantes: Avila Lucas, Pagano Amanda, Liendo Mateo.**Parte 1 - Inventario de puertos y decision de exposicion****25 puntos**

Un grupo de la comision levanto su proyecto en el VPS y corrio `ss -tlnp`. Esta es la salida completa, sin recortar. El proyecto tiene una API en FastAPI, una base MongoDB, un Redis de cache y el panel de Easypanel.

```

root@vps-tup:~# ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Process
LISTEN  0        4096    0.0.0.0:22      users: (("sshd",pid=791))
LISTEN  0        4096    0.0.0.0:80     users: (("traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:443    users: (("traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:3000   users: (("docker-proxy",pid=1704))
LISTEN  0        4096    0.0.0.0:8000   users: (("docker-proxy",pid=1893))
LISTEN  0        4096    0.0.0.0:27017  users: (("docker-proxy",pid=1955))
LISTEN  0        511     127.0.0.1:6379 users: (("redis-server",pid=1512))

```

1.1 Completa el inventario

14 puntos (2 por fila)

Una fila por puerto. En "Rango RFC 6335" escribí si es bien conocido, registrado o dinámico. En "Alcanzable desde internet" respondes lo que debería ser, no lo que hoy es.

Puerto	Proceso	Interfaz en la que escucha	Rango RFC 6335	¿Debe ser alcanzable desde internet?	¿Por qué? (una línea)
22	sshd	0.0.0.0	Bien conocido (0 - 1023)	Si	Para acceder al servidor.
80	Traefik	0.0.0.0	Bien conocido (0 - 1023)	Si	Para obtener HTTPS y para obtener el dominio
443	Traefik	0.0.0.0	Bien conocido (0 - 1023)	Si	Es por donde entra todo el tráfico.
3000	Easypanel	0.0.0.0	Registrado (1024 - 49151)	No	Por que no se debe exponer el puerto directo.
8000	http-alt	0.0.0.0	Registrado (1024–49151)	No	Porque la API no debe recibir tráfico por su puerto interno.
27017	MongoDB	0.0.0.0	Registrado (1024–49151)	No	Solo la necesita la base de datos, no tiene por que verse desde internet.
6379	Redis	127.0.0.1	Registrado (1024 - 49151)	No	Es inalcanzable desde internet aunque el firewall esté apagado

1.2 Elegí la estrategia de cierre

Para los tres servicios que decidiste que no deben verse desde internet pero hoy escuchan en 0.0.0.0, elegí una de las estrategias de la sección 2.8.3 y justifica la respuesta. Ojo: "agregar una regla ufw deny" no es una de ellas, y en la Parte 2 vas a ver por qué.

Explicación de entendimiento: Elegiría publicarlo en local, para permitir un acceso puntual mediante un túnel SSH seguro, evitando la exposición directa a internet y asegurando que ninguna información sensible quede fuera del servidor

Explicación técnica:

- Para el 27017 No publicar el puerto, porque solo la api lo necesita le habla por la red interna de docker.
- Para el 8000 No publicar el puerto, porque el tráfico externo debe entrar por Traefik (443) no directo.
- Para 3000 publicar bajo subdominio propio luego cerrar el puerto el panel sí necesita ser visto desde afuera, pero cifrado y sin exponer el puerto interno.

1.3 Predecir la evidencia externa

(5 puntos)

Suponiendo que el grupo aplica exactamente las correcciones que propusiste, escribí la salida que debería dar `nmap -Pn tudominio.me` corrió desde otra red. Una línea por puerto. Y decime por qué esta comprobación vale más que cualquier otra del capítulo.

Salida esperada (consultamos con claudie porque no sabíamos cómo hacer esa consulta)

Port	Protocol	State	Service
80	TCP	open	http
443	TCP	open	https
22	TCP	open	sshd

La única comprobación confiable consiste en consultar los puertos desde otra máquina. La razón es epistemológica antes que técnica: cualquier herramienta que corra dentro del servidor está describiendo lo que ella cree; solo una prueba desde afuera atraviesa el mismo camino que atravesaría un atacante.

Parte 2 - El recorrido del paquete

15 puntos

Dos paquetes salen de la misma notebook, en la misma red, con un segundo de diferencia. Los dos llegan a la IP publica del mismo servidor. Uno lo filtra `ufw` y el otro no. No es magia ni un error de Ubuntu: es el recorrido.

```
Paquete A  destino 203.0.113.45:22      -> sshd, que corre en el host
Paquete B  destino 203.0.113.45:27017   -> mongo, que corre en un contenedor
                                                publicado con -p 27017:27017
```

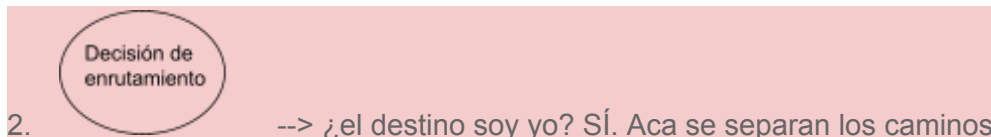
2.1 Traza los dos recorridos

8 puntos (4 por paquete)

Complete la secuencia de puntos de enganche de netfilter que atraviesa cada paquete, desde que entra por la placa de red hasta que llega al proceso. Marca con un círculo el momento exacto en que los dos caminos se separan.

Los cinco puntos de enganche de netfilter:

1. PREROUTING



3. INPUT

4. FORWARD

5. OUTPUT

6. POSTROUTING.

	Paquete A (puerto 22, proceso en el host)	Paquete B (puerto 27017, contenedor publicado)
1	PREROUTING	PREROUTING
2	Decisión de enrutamiento (aquí se separan)	Decisión de enrutamiento (aquí se separan)
3	INPUT	FORWARD
4	Entrega al proceso local (sshd)	POSTROUTING
5		Sale hacia el contenedor (otra máquina)

2.2 Explicalo en una frase de ingeniería

4 puntos

Completa: "sudo ufw deny 27017 no bloquea nada porque ufw escribe sus reglas en la cadena **INPUT**, y el paquete publicado por Docker pasa por la cadena **FORWARD**, de modo que la regla nunca llega a evaluarse para ese tráfico ." – Nunca pasa por **INPUT**. Y las reglas de ufw viven en **INPUT**.

2.3 Qué comando muestra la verdad de adentro

3 puntos

Nombra el comando que permite leer el conjunto de reglas real del núcleo, con las cadenas que creó ufw y las que creó Docker una al lado de la otra. Y explica por qué ese comando, aun siendo más honesto que ufw status, sigue sin ser la verificación definitiva.

El comando es: **sudo nft list ruleset**

Porque sigue siendo una herramienta que corre dentro del servidor. Y "cualquier herramienta que corra dentro del servidor está describiendo lo que ella cree; solo una prueba desde afuera atraviesa el mismo camino que atravesaría un atacante."

`nft list ruleset` te muestra la configuración real y completa (a diferencia de `ufw status`, que sólo muestra sus propias reglas), pero sigue siendo una descripción de reglas, no una prueba de qué paquete efectivamente llega. Solo `nmap -Pn` desde otra red constituye evidencia externa

Parte 3 - El guion de endurecimiento

20 puntos

Doce pasos, desordenados, sobre un servidor recién creado al que hoy se entra con contraseña de root. El objetivo es dejarlo con acceso solo por clave y con el firewall activo, **sin quedarse afuera en el intento**. Este ejercicio es el que mas alumnos pierde en la practica: el orden no es una preferencia de estilo, es la diferencia entre tener servidor y no tenerlo.

```
a) sudo ufw allow OpenSSH
b) sudo ufw enable
c) ssh-keygen -t ed25519 -C "matias@notebook"
d) editar /etc/ssh/sshd_config y poner PasswordAuthentication no
e) sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
f) ssh-copy-id root@203.0.113.45
g) sudo systemctl restart ssh
h) abrir una SEGUNDA terminal y entrar por SSH, sin cerrar la primera
i) sudo apt update && sudo apt install fail2ban -y
j) sudo ufw status verbose
k) nmap -Pn tudominio.me (desde otra maquina, en otra red)
l) ssh root@203.0.113.45 y comprobar que NO pide contraseña
```

3.1 Ordena los doce pasos

10 puntos

Escribí las doce letras en el orden correcto de ejecución.

1. **c)** `ssh-keygen -t ed25519 -C "matias@notebook"`
2. **f)** `ssh-copy-id root@203.0.113.45`
3. **h)** abrir una SEGUNDA terminal y entrar por SSH, sin cerrar la primera
4. **d)** editar `/etc/ssh/sshd_config` → `PasswordAuthentication no`
5. **g)** `sudo systemctl restart ssh`
6. **l)** `ssh root@203.0.113.45` y comprobar que NO pide contraseña
7. **a)** `sudo ufw allow OpenSSH`
8. **e)** `sudo ufw allow 80/tcp && sudo ufw allow 443/tcp`
9. **b)** `sudo ufw enable`
10. **j)** `sudo ufw status verbose`
11. **k)** `nmap -Pn tudominio.me` (desde otra máquina, en otra red)
12. **i)** `sudo apt update && sudo apt install fail2ban -y`

Primero hay que generar el par de claves (en tu equipo, nunca en el servidor).

Copiamos la clave pública al servidor mientras la contraseña todavía funciona (`ssh-copy-id` la necesita).

Verificar que la clave funciona en una segunda terminal, sin cerrar la primera (esta es la red de seguridad que está en el apunte).

Recién ahí podemos deshabilitar la contraseña y reiniciar el servicio. Confirmamos que ya no pide contraseña. Armamos el firewall: **primero se permite el 22**, después 80/443, **recién al final** `ufw enable`. Verificamos el estado (interno) y después hay que instalar fail2ban. Finalmente se termina con la única prueba externa real: `nmap` desde otra red.

3.2 Marca los tres puntos de no retorno

6 puntos (2 por punto)

Tres de esos pasos, ejecutados fuera de orden, te dejan afuera del servidor y solo se arreglan por la consola de emergencia de Hostinger. Identificalos y decime en cada caso qué error concreto los convierte en trampa.

Paso	Error si ejecuta fuera de orden
g) <code>sudo systemctl restart ssh</code>	Si se corre antes de verificar en la segunda terminal (h) que la clave funciona, y la clave no quedó bien instalada, el reinicio aplica PasswordAuthentication no y te quedás sin ninguna forma de entrar: ni contraseña, ni clave.
f) <code>ssh-copy-id</code>	Si se intenta ejecutar después de <code>d)</code> (deshabilitar contraseña), ya no hay forma de copiar la clave: <code>ssh-copy-id</code> necesita autenticarse con contraseña para instalar la clave pública, y esa vía ya estaría cerrada. Es un problema de huevo y gallina.
b) <code>sudo ufw enable</code>	Si se corre antes de <code>a)</code> (<code>allow OpenSSH</code>), el firewall pasa a denegar todo el tráfico entrante por defecto — incluida tu propia sesión SSH y te cortas vos mismo el acceso en el acto.

3.3 La red de seguridad

4 puntos

Dos de los doce pasos no cambian absolutamente nada en el servidor: solo miran. Identificalos y explica que principio de trabajo representan y por que van exactamente donde los pusiste.

- **j) `sudo ufw status verbose`**
- **k) `nmap -Pn tudominio.me`**

Son lecturas de estado, representan la verificación antes de avanzar “la distinción entre lo que una herramienta declara y lo que realmente pasa ”

j) te dice lo que `ufw` cree que está pasando (evidencia interna), y k) es la única prueba *externa* real, la que de verdad certifica que el servidor está cerrado como corresponde.

Parte 4 - Auditoria de un archivo de configuracion**15 puntos**

Un grupo entrego este fragmento de `/etc/ssh/sshd_config` como "servidor endurecido". Tiene **cinco problemas reales y una línea que parece un problema y no lo es**.

```
# /etc/ssh/sshd_config (fragmento entregado por un grupo)
1  Port 22
2  PermitRootLogin yes
3  PubkeyAuthentication yes
4  PasswordAuthentication yes
5  PermitEmptyPasswords yes
6  MaxAuthTries 30
7  X11Forwarding yes
8  AuthorizedKeysFile .ssh/authorized_keys
9  UsePAM yes
```

4.1 Los cinco problemas

10 puntos (2 por problema)

Para cada uno: número de línea, valor corregido y una línea de justificación. Sin justificación no suma.

Línea	Original	Corregido	Justificación
2	PermitRootLogin yes	PermitRootLogin prohibit-password	Permite entrar como root con contraseña, el valor correcto permite solo root por clave nunca por contraseña.
4	PasswordAuthentication yes	PasswordAuthentication no	Deja la puerta abierta para la fuerza bruta.
5	PermitEmptyPasswords yes	PermitEmptyPasswords no	Permite contraseñas vacías, es la más grave de las cinco. Es el ejemplo extremo del "valor predeterminado permisivo"
6	MaxAuthTries 30	MaxAuthTries 3 (o similar que sea bajo)	Con 30 intentos por conexión, se facilita muchísimo la fuerza bruta automatizada, ("un solo equipo puede emitir del orden de un millón de paquetes por segundo"). Un número bajo corta la sesión antes.
7	X11Forwarding yes	X11Forwarding no	Habilita reenvío de interfaz gráfica, algo que un servidor sin entorno gráfico no necesita nunca. Viola mínimo privilegio. Superficie de ataque innecesaria para una función que no se usa.

4.2 La línea que no es un problema

5 puntos

Un integrante propone cambiar la línea 1 a Port 2222 "para que no lo encuentren los bots". Identifica por qué esa medida no mejora la seguridad del servidor en términos del capítulo, y nombra además el riesgo operativo concreto que introduce si se aplica en el orden equivocado.

La asignación es una convención, no una imposición técnica cambiar el número de puerto es **seguridad por oscuridad**, no una de las tres medidas reales de endurecimiento (autenticación por clave, firewall, fail2ban). No cambia nada respecto a la fuerza bruta dirigida ni a un escaneo completo de puertos, barre los 65.535 puertos de una IP en minutos 2222 se encuentra exactamente igual que el 22.

Con el riesgo de que si cambiás el puerto de SSH y no actualizas la regla de ufw correspondiente antes de reiniciar el servicio (o intentas reconectarte sin especificar -p 2222 en el comando ssh), te quedás afuera del servidor exactamente igual que en los "puntos de no retorno" de la Parte 3, con el agravante de que ni siquiera hiciste algo que aportará seguridad real a cambio del riesgo.

Parte 5 - Peritaje: reconstruir un incidente

15 puntos

Un grupo pide ayuda: "nos borraron una tabla de la base y no entendemos como, si el SSH estaba blindado". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
root@vps-tup:~# docker ps --format "{{.Names}}\t{{.Ports}}"
tup-api      0.0.0.0:8000->8000/tcp
tup-db       0.0.0.0:5432->5432/tcp
tup-front    0.0.0.0:8080->80/tcp

root@vps-tup:~# sudo ufw status
Estado: activo
Hasta      Accion      Desde
22/tcp     ALLOW IN    Anywhere
80/tcp     ALLOW IN    Anywhere
443/tcp    ALLOW IN    Anywhere
5432/tcp    DENY IN     Anywhere

--- desde la notebook de un companero, en otra red ---
$ nmap -Pn 203.0.113.45
PORT      STATE  SERVICE
22/tcp    open   ssh
80/tcp    open   http
443/tcp   open   https
5432/tcp  open   postgresql
8000/tcp  open   http-alt
8080/tcp  open   http-proxy

root@vps-tup:~# grep -c "Failed password" /var/log/auth.log
0

--- registro del contenedor de la base ---
2026-08-14 03:12:07 UTC [4412] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:09 UTC [4415] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:11 UTC [4419] FATAL: password authentication failed for user "postgres"
2026-08-14 03:14:41 UTC [4602] LOG: connection authorized: user=postgres database=tup
2026-08-14 03:19:55 UTC [4602] LOG: statement: DROP TABLE operaciones;
```

5.1 Reconstruye la cadena de hechos

5 puntos

En cinco líneas como máximo, contame que paso, en orden, desde que el grupo levantó el proyecto hasta el DROP TABLE. Sostiene cada afirmación en una línea concreta de la evidencia.

Basándonos en los textos y filminas de la unidad sobre el recorrido de los paquetes y el problema de Docker con el firewall hacemos esta reconstrucción de los hechos.

1. El grupo levantó el proyecto publicando directamente con Docker los puertos de la API (8000), la base (5432) y el frontend (8080), quedando los tres expuestos a cualquier IP.
2. Para "proteger" la base, agregaron una regla ufw deny 5432, que en la práctica no sirvió de nada porque el tráfico hacia un puerto publicado por Docker no pasa por donde ufw filtra.
3. Por eso ufw status muestra el 5432 como bloqueado mientras que un escaneo hecho desde otra red lo encuentra abierto: son dos verdades distintas, una interna (lo que ufw cree) y otra externa (lo que realmente llega).
4. Un atacante aprovechó esa exposición y probó credenciales contra Postgres: tres intentos fallidos y, minutos después, una conexión exitosa como usuario postgres.
5. Una vez adentro, no necesitó nada más que ese acceso directo a la base para borrar la tabla de operaciones.

5.2 Las dos contradicciones aparentes

4 puntos (2 cada una)

La evidencia parece contradecirse dos veces. Explica cada caso: (i) ufw status dice que el 5432 está bloqueado y nmap lo ve abierto; (ii) hubo una intrusión y sin embargo auth.log no registra un solo intento fallido.

(i) ufw dice "bloqueado" y nmap dice "abierto": no es contradicción real, es exactamente el problema central lo que venimos viendo en todo el capítulo "Docker no respeta el firewall". ufw status solo refleja sus propias reglas en INPUT; no dice nada sobre lo que realmente llega al proceso. nmap desde afuera atraviesa el camino real del paquete (PREROUTING → nat de Docker → FORWARD) y por eso ve la verdad: el puerto está efectivamente alcanzable.

(ii) hubo intrusión pero auth.log no registra fallos: tampoco es contradicción, es una consecuencia de dónde entró el atacante. auth.log registra intentos contra SSH (el servicio del host). Acá el atacante nunca tocó SSH: entró directo al contenedor de Postgres, cuyos intentos fallidos quedan en el log del propio contenedor de la base (que sí muestra los tres FATAL: password authentication failed), no en auth.log del sistema operativo. Son dos superficies de ataque completamente distintas

5.3 Los principios violados

3 puntos

De los tres principios de la sección 2.12.1, indica cuales se violaron y en qué decisión concreta del grupo.

1. **Valores predeterminados seguros:** violado al publicar 5432 y 8000 con -p 0.0.0.0:... en vez de partir de "nada expuesto salvo lo estrictamente necesario". Es el mismo error que el apunte crítica en Redis/MongoDB
2. **Mínimo privilegio:** violado porque ni la base de datos ni la API necesitan ser alcanzables directamente desde internet solo el front y, a través de él o de Traefik, la API. Publicar 5432 y 8000 les da acceso a mucho más de lo que su función requiere.
3. **Economía del mecanismo:** violado al confiar en una regla de ufw (deny 5432) en lugar de la solución simple y verificable: no publicar el puerto. La regla agrega complejidad y una falsa sensación de seguridad, sin cerrar nada de verdad.

5.4 Las tres correcciones, en orden

3 puntos

Escribí las tres primeras medidas que aplicarías, en el orden en que las aplicarías. Justifica el orden, no solo el contenido.

1. **Dejar de publicar los puertos 5432 y 8000.** Sacar el mapeo -p del docker run o del compose y recrear los contenedores para que dejen de escuchar en 0.0.0.0. Va primero porque es la causa raíz de todo el incidente: mientras el puerto siga publicado, cualquier otra medida es solo maquillar el problema, que vuelve a aparecer apenas alguien intente entrar de nuevo.
2. **Cambiar la contraseña del usuario postgres.** Ya que la credencial actual ya quedó comprometida, porque el log muestra una conexión exitosa desde una IP que no es la del grupo. Va segundo porque, aunque el puerto ya no esté expuesto, no tiene sentido confiar en una contraseña que un atacante ya usó una vez.
3. **Verificar con nmap -Pn desde otra red que 5432 y 8000 ya no figuran como abiertos.** Va al final porque es la única forma de comprobar que la corrección realmente funcionó. No alcanza con mirar ufw status ni la configuración del docker-compose, porque ninguna de las dos cosas garantiza lo que en verdad llega desde afuera.

Parte 6 - El plan de puertos de tu propio proyecto

10 puntos

Esta parte no es un ejercicio de laboratorio: es el plano de lo que vas a desplegar en las Clases 4 y 5. El dominio ya lo tienes de la Clase 1; el servidor puede estar o no estar, no importa. Lo que define ahora es que se publica y que no, antes de escribir un solo docker run.

Servicio	Subdominio que va a usar	Puerto interno	¿Se publica al mundo?	¿Cómo llega el tráfico?	Registro DNS necesario
Panel de administración	easypanel.tudominio.me	3000	No (directo)	easypanel. dominio.me https	easypanel. dominio.me o cubre el comodín * dominio.me
Frontend	tudominio.me (o www.tudominio.me)	Interno como el 80	Si	escribiendo www.dominio.me o dominio.me	Ninguno adicional está cubierto por el registro @ (raíz) o el comodín (si es www.)
API	api.tudominio.me	Interno como el 8000	Si	app web o api.dominio.me	Ninguno adicional lo cubre el comodín *
Base de datos	sin subdominio	Interno como el 5432	No	Desde la API	Ninguno

6.1 El registro que te ahorra trabajo

5 puntos

De todos los registros DNS de tu tabla, ¿cuántos tuviste que crear a mano en la Clase 1 y cuántos te resuelve un solo registro? Nombra y explica qué problema evita cuando en la Clase 5 agregues un servicio nuevo.

De los cuatro servicios, el único registro que se cargó a mano fue el comodín (* → IP del VPS, tipo A). Ese único registro resuelve automáticamente cualquier subdominio presente o futuro, api., easypanel., o lo que agreguen en la Clase 5 sin volver a tocar el panel DNS.

el comodín no cubre el dominio raíz (tudominio.me sin nada adelante), por eso necesitaron el segundo registro con nombre @. Ese sí lo tenemos que cargar aparte, una sola vez, y no por cada servicio.

6.2 La línea que te vas a agradecer

5 puntos

Mira tu tabla y responde: si mañana alguien de tu grupo levanta la base "un ratito con -p 5432:5432 para probar con DB Beaver desde su casa", ¿que le decis? Escribe la respuesta técnica y la alternativa concreta que le ofreces, porque la necesidad es legítima.

Respuesta técnica: No, no conviene. Aunque parezca algo temporal y controlado, un contenedor levantado con -p 5432:5432 no comparte la pila de red del anfitrión: tiene su propia dirección IP, y desde el punto de vista del núcleo es otra máquina alcanzable a través del servidor. Ese tráfico entra por PREROUTING, donde Docker ya reescribió el destino, y sigue por FORWARD nunca pasa por INPUT, que es donde vive el firewall. Entonces aunque el firewall esté activo, este puerto queda accesible desde cualquier lugar de internet. Y no hace falta que nadie lo encuentre a propósito.

El espacio completo de direcciones IPv4 se puede escanear en minutos, y existen buscadores como Shodan o Censys que ya tienen indexado qué servicio corre en cada dirección del mundo. Un servidor recién expuesto empieza a recibir intentos de acceso dentro de la primera hora.

La alternativa: usar un túnel SSH. Con `ssh -L 5432:127.0.0.1:5432 root@tudominio.com`, así se conecta desde DBEaver a `localhost:5432` en su propia máquina, y esa conexión viaja cifrada por SSH hasta el servidor. Ahí, para Postgres, la conexión aparenta ser local, nunca hizo falta abrir el puerto hacia afuera. Es la aplicación práctica de la misma idea que explica por qué escuchar en `127.0.0.1` es la forma más barata y más robusta de cerrar algo: si nadie escucha del lado de afuera, no hay firewall que valga porque no hace falta ninguno.

Cierre - Para discutir en la mesa

sin puntaje

La conclusión del capítulo es una sola frase: **el puerto más seguro es el que nunca se publica**. No el que está bloqueado, no el que tiene contraseña fuerte, no el que está en un número raro. El que no existe hacia afuera.

Discutan cinco minutos, sin escribir: en el código que ya escribieron en esta carrera, donde están sus "puertos publicados de mas"? Un endpoint que devuelve más campos de los que la pantalla usa. Un usuario de base con permisos de administrador porque era más rápido. Un repositorio público con un archivo `.env` adentro. Una variable de entorno que llegó al bundle del frontend. En cada uno de esos casos, cuál sería el equivalente a correr `nmap` desde afuera?

Lo que se entrega

Las hojas del taller, con el nombre de los tres integrantes en la primera. Si la Parte 6 les quedó a medias, entreguenla igual como está: esa tabla es el plano del despliegue de las Clases 4 y 5, y la vamos a retomar. Lo que **no** se entrega en blanco es la justificación: prefiero una decisión discutible bien fundamentada que una correcta sin razón.

Nosotros todavía no hicimos frontend, pero basándonos en lo leído y en las clases que tuvimos, creemos que todo gira en torno a la misma distinción: lo que uno cree que está protegido versus lo que realmente es alcanzable desde afuera. `ufw` status miente, `nmap` no. En los cuatro casos el error es siempre el mismo: confiar en que la capa que vos controlas decide qué se ve, ya sea la pantalla, el nombre del usuario, el `.gitignore`, o la palabra "environment variable" en vez de verificar qué es efectivamente alcanzable desde el lado de quien no confía en nada de eso. Según lo que vimos, hay que tener mucho cuidado al momento de levantar una página o una aplicación, porque un error acá te puede salir carísimo, literalmente. Lo que estuvimos charlando en clase, si el servidor queda comprometido te pueden usar para minar criptomonedas y terminás pagando una factura que no esperabas, o peor, no pudiendo pagarla. También te hace entender que hay todo un mundo de gente buscando descuidos ajenos para aprovecharse, todo el tiempo, de forma automática, no hace falta que nadie te ataque a propósito. En fin, tenemos que ser cuidadosos y responsables, pero sin dejar de animarnos a construir cosas porque de los errores, bien entendidos, es de donde se aprende a ser mejores programadores.